

DnCNN reproduces to within 0.19 dB, at a cost of 44 GPU-hours

The baseline is trustworthy. Getting there exposed a data loader that wasted nine tenths of every training step, now fixed and 4.8× faster.

Model DnCNN, 17 layers, width 64, 558,403 params

Noise AWGN $\sigma=25$

Checkpoint step 286,000

Date 2 September 2026

CBSD68, CROSS-DATASET 31.04_{dB} Published DnCNN colour: 31.23 dB. Gap of 0.19 dB.	DIV2K VAL, FULL IMAGES 32.65_{dB} Exit test asked for roughly 29 dB. Clears it by 3.6 dB.
LOADER THROUGHPUT 4.8_× faster 1.79 → 8.63 steps/s. A 300k run: 46.7 h → 9.7 h.	

What was measured

Three PSNR figures circulate for this one checkpoint, and they are easy to confuse because all three are honest measurements of different things. Only the last row belongs next to a published number.

PROTOCOL	IMAGES	NOISY IN	RESTORED OUT	GAIN
DIV2K val, 128px crops (training log)	16	20.56	34.75	+14.19
DIV2K val, full images	100	20.70	32.65	+11.95
CBSD68, full images	68	20.53	31.04	+10.51

Rows two and three are appended to `results/benchmark.csv`. Row one is the in-loop validation that `train.py` writes every 2,000 steps.

The measured noise floor confirms the degradation pipeline is correct. For $\sigma=25$ in 0–255 units the theoretical input PSNR is 20.17 dB, and clipping to

[0,1] lifts it slightly. All three protocols land between 20.53 and 20.70 dB.

The exit test named the wrong number

`plan.md` sets the Phase 1 bar at “roughly 29 dB PSNR on the DIV2K validation set”. That figure is the *grayscale* BSD68 result for DnCNN-S. This model is `DnCNN(channels=3)`, so the comparable published figure is the colour CBSD68 one, about 31.23 dB.

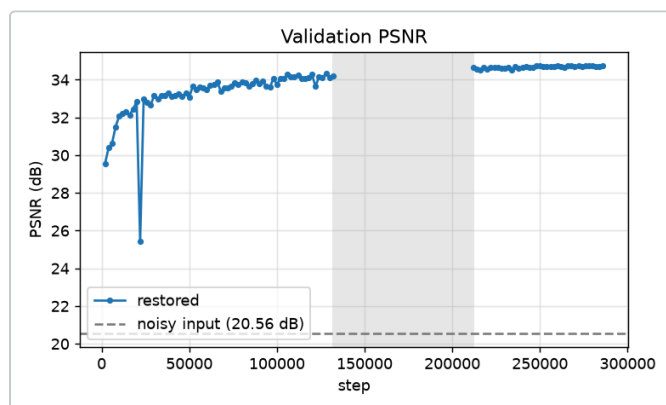
VERDICT

At 31.04 dB the baseline sits 0.19 dB below published, well inside the 0.5 dB band the plan requires. Phase 1 passes for DnCNN. Correct the target in `plan.md` so the next reader is not comparing colour results against a grayscale reference.

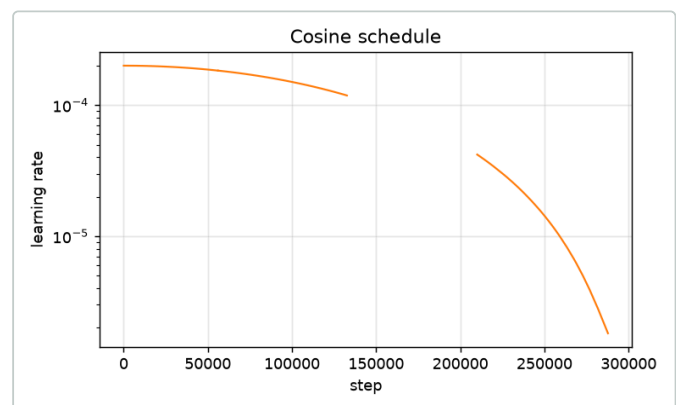
Verify 31.23 dB against Zhang et al., TIP 2017 before it goes in the report. It is quoted here from memory, not from the paper.

The run had converged before it stopped

Training halted at step 286,000 of a scheduled 300,000 when the Kaggle quota ran out. That truncation costs nothing measurable, and the schedule itself is the argument.



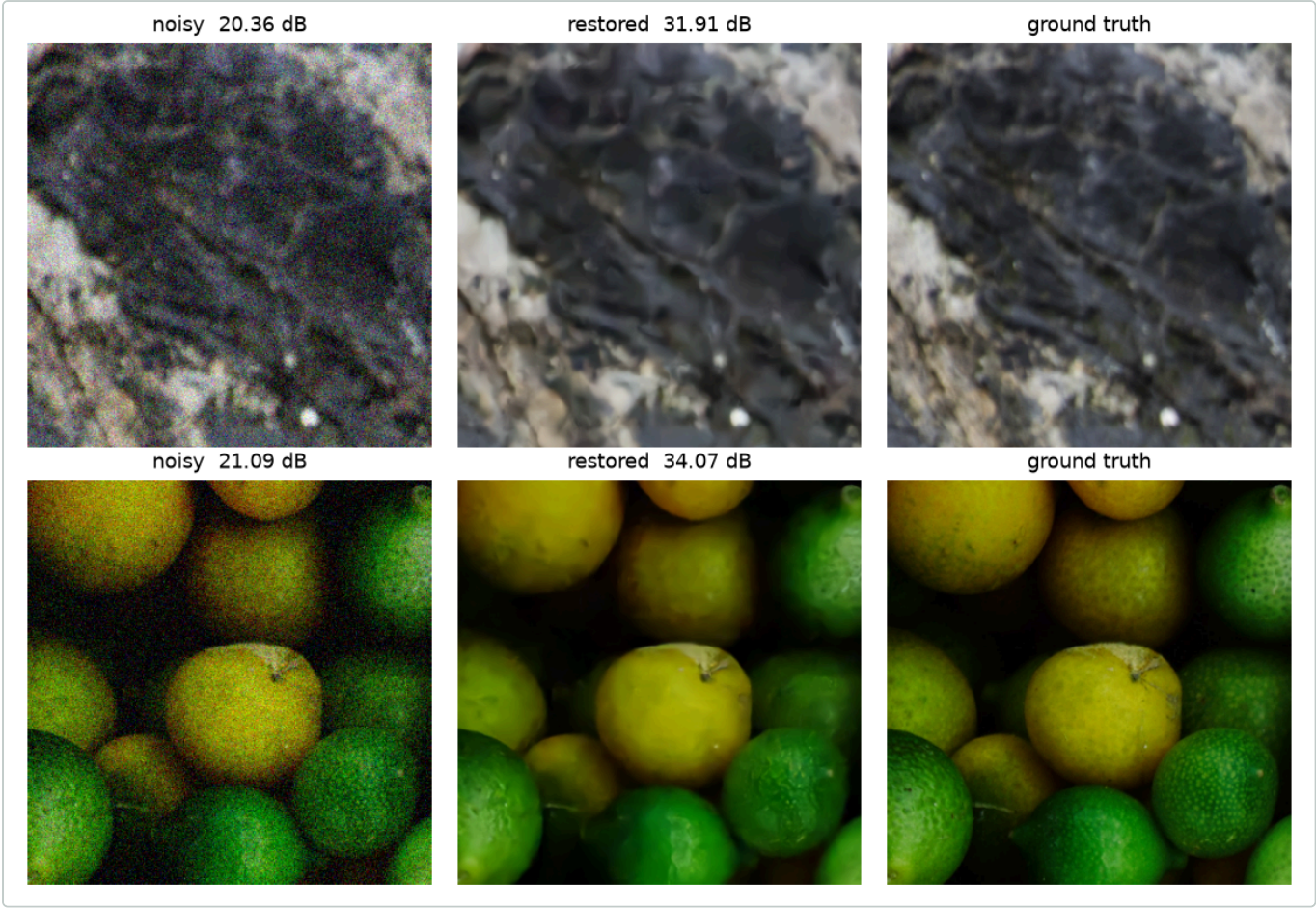
Validation PSNR. Flat within ± 0.04 dB across the final 22,000 steps. Best was 34.75 dB at step 280,000. The shaded band is a hole in the log, not a hole in training.



Cosine schedule. At step 286,000 the learning rate is $2.07e-6$, one percent of peak. The remaining 14,000 steps carry 0.06% of the run's integrated learning rate.

Two independent signals agree, so the checkpoint at 286,000 is the final baseline and the run should not be resumed. Spending 2.2 hours of a 30-hour weekly quota to reach a round number would buy noise.

What it looks like

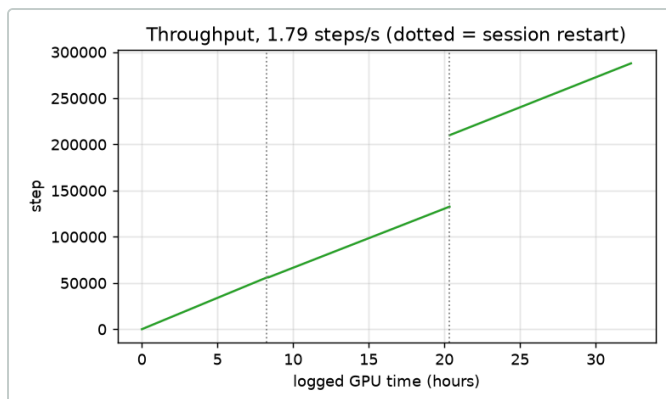


Noisy, restored, ground truth. 320px crops from DIV2K validation at $\sigma=25$. Denoising is clean on smooth regions; fine texture is softened, which is the known DnCNN failure mode and the gap FCSG-Net's high-frequency expert is meant to close.

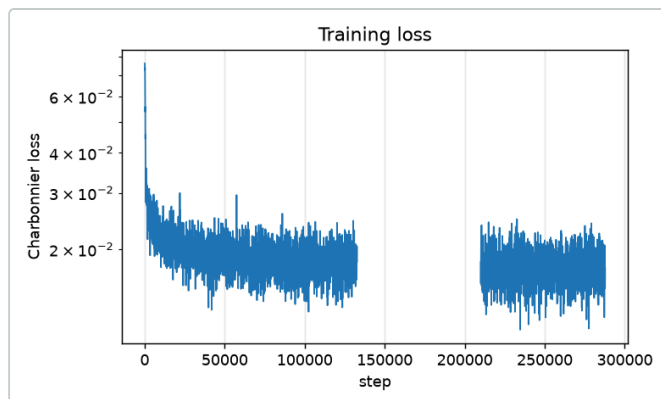
What it cost, and why that matters more than the result

The baseline consumed roughly 44 GPU-hours across four Kaggle sessions, at a steady 1.79 steps per second. A full 300,000-step run costs 46 hours at that rate, against a 30-hour weekly quota.

SESSION	STEPS	HOURS	RATE
1	50 – 56,600	8.34	1.88/s
2	56,050 – 132,650	11.98	1.78/s
3 (log lost)	132,650 – 210,050	~12.0	—
4	210,050 – 287,750	11.98	1.80/s



Throughput. Dotted lines mark session restarts. The break is session 3, whose `train_log.csv` was never seeded on resume, so 77,400 steps and about 12 hours are absent from the record.



Training loss. Charbonnier, 0.0761 down to 0.0174 over 4,220 logged points. No instability, no divergence after any resume.

The diagnosis

`PatchDataset.__getitem__` opened and fully decoded a 2040×1356 PNG to take a single 128px crop, then discarded the decode. A 0.56M-parameter DnCNN at batch 16 should take tens of milliseconds per step on a P100; it was taking 555. Roughly nine tenths of every step was libpng, not CUDA.

WHY THIS BLOCKS PHASE 4

FCSG-Net is larger than DnCNN, and the ablation table is several runs. At 46 hours each, on 30 GPU-hours per week, the table alone would take more than a month of wall-clock time. The loader was the constraint on the research schedule, not the GPU. It has since been fixed, and the measurement is below.

The fix

`training/build_tiles.py` extracts crops once into a `uint8` NumPy memmap, and `TileDataset` serves them with no decode at all. At 64 crops per image that is 51,200 tiles, about 2.5 GB, times eight flip and rotation variants.

```
python training/build_tiles.py --data /content/DIV2K \
    --out tiles.npy --patch 128 --per-image 64
```

```
python training/train.py --config configs/dncnn.toml \
    --data /content/DIV2K --tiles tiles.npy --out ckpt
```

LOADER	RATE	300K RUN	SESSIONS NEEDED
PNG decode per sample	1.79/s	46.7 h	4
Tile memmap	8.63/s	9.7 h	1

Measured on a Colab T4 over 2,000 steps, same model and batch size. The run now fits inside one session under the 11-hour budget, which removes the resume cycle that lost session 3's log in the first place.

CAVEAT

8.63 steps/s is a DnCNN number. FCSG-Net carries an FFT, nine experts, and a router, so it will be slower and the bottleneck may return to the GPU where it belongs. Re-time once the model exists, before committing to a step count for Phase 3.

Open items

- **FFDNet is not written.** The Phase 1 exit test has two halves and only DnCNN is done. `models/ffdnet.py` does not exist, and `build_model` in `train.py` has no branch for it. This is code, not compute, and it blocks M1.
- **`notes/related.md` is an empty skeleton.** Four headings, no paragraphs. `plan.md` requires MWCNN, SFNet/FSNet, and the sparse mixture-of-experts paper to be read before M1, because the novelty claim is positioned against SFNet.
- **Set `steps` deliberately for Phase 3.** At 8.63 steps/s a 300,000-step DnCNN run takes 9.7 hours, which fits one session with about an hour of margin. FCSG-Net will be slower, so re-time it before choosing its step count.
- **Session 3's log is unrecoverable.** Seed `train_log.csv` from the previous output on every resume, or the record keeps losing sessions.
- **The CBSD68 qualitative figure was overwritten** by the DIV2K evaluation run. Re-run that evaluation if both strips are wanted.

Generated from `checkpoints/train_log.csv` (4,324 rows) and `results/benchmark.csv`. Figures regenerate with `python evaluation/plots.py --csv checkpoints/train_log.csv --out results/figures`.